

Java Collections Framework

Lists, Sets, Maps, Queues & the Stream API — with Examples

March 2026

1. Introduction

The Java Collections Framework (JCF), introduced in Java 2 (JDK 1.2), provides a unified architecture for storing and manipulating groups of objects. It defines interfaces, abstract implementations, and concrete classes that cover the most common data-structure needs.

Before the JCF, Java only had ad-hoc classes like `Vector`, `Hashtable`, and arrays. The framework unified these, added generic type safety (Java 5), and continues to grow — most recently with sequenced collections (Java 21).

Core interfaces

- `Collection<E>` — root interface; basic add/remove/contains/size
- `List<E>` — ordered, indexed, allows duplicates
- `Set<E>` — no duplicates
- `Queue<E>` / `Deque<E>` — FIFO / double-ended queues
- `Map<K,V>` — key-value pairs (not a `Collection` subtype)
- `Iterable<E>` — enables for-each loops

2. Interface Hierarchy

The diagram below (described in text) shows how the core interfaces relate:

`Iterable`

`Collection`

`List` (`ArrayList`, `LinkedList`, `Vector`, `Stack`, `CopyOnWriteArrayList`)

`Set` (`HashSet`, `LinkedHashSet`, `TreeSet`, `CopyOnWriteArraySet`)

`SortedSet` → `NavigableSet`

`Queue` (`LinkedList`, `PriorityQueue`, `ArrayDeque`)

`Deque` (`ArrayDeque`, `LinkedList`)

`Map` (separate hierarchy)

`HashMap`, `LinkedHashMap`, `Hashtable`

`SortedMap` → `NavigableMap` → `TreeMap`

`ConcurrentHashMap` (thread-safe)

3. List

A List is an ordered collection that permits duplicates and provides positional (index-based) access.

3.1 ArrayList

Attribute	Detail
Backing store	Resizable array
Access	O(1) random access by index
Insert/Delete	O(n) for middle insertions; O(1) amortised at tail
Thread-safe	No — use <code>Collections.synchronizedList()</code> or <code>CopyOnWriteArrayList</code>
When to use	Default List choice; mostly read/append workloads

```
import java.util.*;

List<String> fruits = new ArrayList<>();
fruits.add("Apple");
fruits.add("Banana");
fruits.add("Cherry");
fruits.add(1, "Blueberry"); // insert at index 1
System.out.println(fruits.get(0)); // Apple
System.out.println(fruits.size()); // 4
fruits.remove("Banana"); // remove by value
fruits.remove(0); // remove by index

// Iterate
for (String f : fruits) System.out.println(f);

// Sort
Collections.sort(fruits);
fruits.sort(Comparator.reverseOrder());

// Convert array to list (fixed-size)
List<String> fixed = Arrays.asList("X", "Y", "Z");

// Immutable list (Java 9+)
List<String> immutable = List.of("A", "B", "C");
```

3.2 LinkedList

Attribute	Detail
-----------	--------

Backing store	Doubly-linked list
Access	$O(n)$ — no random access
Insert/Delete	$O(1)$ at head or tail; $O(n)$ in middle (finding node)
Also implements	Deque<E> — usable as stack or queue
When to use	Frequent insert/delete at ends; implementing a queue or stack

```

LinkedList<Integer> list = new LinkedList<>();
list.addFirst(1); // [1]
list.addLast(3); // [1, 3]
list.add(1, 2); // [1, 2, 3]
System.out.println(list.getFirst()); // 1
System.out.println(list.getLast()); // 3
list.removeFirst(); // [2, 3]
// Use as a Queue (FIFO)
Queue<String> queue = new LinkedList<>();
queue.offer("task1");
queue.offer("task2");
String next = queue.poll(); // removes & returns "task1"
// Use as a Stack (LIFO)
Deque<String> stack = new LinkedList<>();
stack.push("a");
stack.push("b");
System.out.println(stack.pop()); // b

```

3.3 List Comparison

Feature	ArrayList	LinkedList
Random access (get)	$O(1)$	$O(n)$
Insert at tail	$O(1)$ amortised	$O(1)$
Insert in middle	$O(n)$	$O(n)$ to find, $O(1)$ to link
Memory	Lower (array)	Higher (node pointers)
Implements Deque	No	Yes
Typical choice	Default	Only for queue/deque use

4. Set

A Set contains no duplicate elements. The contract: for any e1 and e2, e1.equals(e2) must not both be in the set.

4.1 HashSet

Attribute	Detail
Backing store	HashMap internally
Order	No guaranteed order
Performance	O(1) add/remove/contains (average)
Null	Permits one null
When to use	Fast membership testing; order doesn't matter

```
Set<String> set = new HashSet<>();
set.add("Java");
set.add("Python");
set.add("Java"); // duplicate — ignored
System.out.println(set.size()); // 2
System.out.println(set.contains("Java")); // true
// Set operations
Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4));
Set<Integer> b = new HashSet<>(Set.of(3, 4, 5, 6));
Set<Integer> union = new HashSet<>(a); union.addAll(b); // {1,2,3,4,5,6}
Set<Integer> inter = new HashSet<>(a); inter.retainAll(b); // {3,4}
Set<Integer> diff = new HashSet<>(a); diff.removeAll(b); // {1,2}
```

4.2 LinkedHashSet

Maintains insertion order. Slightly slower than HashSet due to doubly-linked list bookkeeping, but iteration order is predictable.

```
Set<String> ordered = new LinkedHashSet<>();
ordered.add("Banana");
ordered.add("Apple");
ordered.add("Cherry");
System.out.println(ordered); // [Banana, Apple, Cherry] — insertion order preserved
```

4.3 TreeSet

Attribute	Detail
Backing store	Red-Black tree (TreeMap internally)
Order	Natural order or custom Comparator
Performance	$O(\log n)$ add/remove/contains
Null	Does NOT permit null
When to use	Need sorted iteration or range queries

```
TreeSet<Integer> nums = new TreeSet<>();
nums.addAll(List.of(5, 2, 8, 1, 9, 3));
System.out.println(nums); // [1, 2, 3, 5, 8, 9] — sorted
System.out.println(nums.first()); // 1
System.out.println(nums.last()); // 9
System.out.println(nums.floor(4)); // 3 (largest <= 4)
System.out.println(nums.ceiling(4)); // 5 (smallest >= 4)
System.out.println(nums.subSet(2, 6)); // [2, 3, 5]
// Custom comparator — reverse order
TreeSet<String> rev = new TreeSet<>(Comparator.reverseOrder());
rev.addAll(List.of("b", "a", "c"));
System.out.println(rev); // [c, b, a]
```

4.4 Set Comparison

Feature	HashSet	LinkedHashSet	TreeSet
Order	None	Insertion	Sorted
add/contains	$O(1)$ avg	$O(1)$ avg	$O(\log n)$
Null	One allowed	One allowed	Not allowed
Memory	Low	Medium	Medium
Range queries	No	No	Yes

5. Queue & Deque

Queue models a FIFO data structure. Deque (double-ended queue) allows insert/remove at both ends, making it work as both a queue and a stack.

5.1 PriorityQueue

Attribute	Detail
Order	Natural order or custom Comparator (min-heap by default)

Performance	$O(\log n)$ offer/poll; $O(1)$ peek
Null	Not permitted
When to use	Task scheduling by priority, Dijkstra's algorithm, top-N problems

```

PriorityQueue<Integer> minHeap = new PriorityQueue<>();
minHeap.addAll(List.of(5, 1, 3, 2, 4));
while (!minHeap.isEmpty()) {
    System.out.print(minHeap.poll() + " "); // 1 2 3 4 5
}
// Max-heap
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());
maxHeap.addAll(List.of(5, 1, 3, 2, 4));
System.out.println(maxHeap.peek()); // 5
// Custom objects
record Task(String name, int priority) {}
PriorityQueue<Task> tasks = new PriorityQueue<>(
    Comparator.comparingInt(Task::priority)
);
tasks.offer(new Task("Low-priority", 3));
tasks.offer(new Task("High-priority", 1));
tasks.offer(new Task("Med-priority", 2));
System.out.println(tasks.poll().name()); // High-priority

```

5.2 ArrayDeque

Attribute	Detail
Backing store	Resizable circular array
Performance	$O(1)$ push/pop/peek at both ends
Null	Not permitted
Versus Stack	Faster than <code>java.util.Stack</code> ; prefer <code>ArrayDeque</code> as a stack
When to use	Stack, queue, or sliding-window problems

```

Deque<String> deque = new ArrayDeque<>();
// Use as Stack (LIFO)
deque.push("first");

```

```

deque.push("second");
System.out.println(deque.pop()); // second
// Use as Queue (FIFO)
deque.offerLast("A");
deque.offerLast("B");
System.out.println(deque.pollFirst()); // A
// Sliding window maximum — classic ArrayDeque use case
// Store indices of useful elements in decreasing order
int[] arr = {1, 3, -1, -3, 5, 3, 6, 7};
int k = 3;
Deque<Integer> window = new ArrayDeque<>();
for (int i = 0; i < arr.length; i++) {
    if (!window.isEmpty() && window.peekFirst() < i - k + 1)
        window.pollFirst();
    while (!window.isEmpty() && arr[window.peekLast()] < arr[i])
        window.pollLast();
    window.offerLast(i);
    if (i >= k - 1) System.out.print(arr[window.peekFirst()] + " ");
}
// Output: 3 3 5 5 6 7

```

6. Map

A Map stores key-value pairs. Keys are unique; values can be duplicated. Map is NOT a subtype of Collection.

6.1 HashMap

Attribute	Detail
Backing store	Hash table (array of buckets + linked list / tree per bucket)
Order	No guaranteed order
Performance	O(1) average get/put; O(n) worst case (hash collision)
Null	One null key; multiple null values allowed
When to use	Default Map; fast lookups by key

```

Map<String, Integer> scores = new HashMap<>();
scores.put("Alice", 95);

```

```

scores.put("Bob", 87);
scores.put("Carol", 92);
System.out.println(scores.get("Alice")); // 95
System.out.println(scores.getDefault("Dan", 0)); // 0
// Conditional put
scores.putIfAbsent("Dave", 80);
scores.merge("Alice", 5, Integer::sum); // Alice = 100
// compute — update based on existing value
scores.compute("Bob", (k, v) -> v == null ? 1 : v + 10); // Bob
= 97
// Iterate
for (Map.Entry<String, Integer> e : scores.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
// Java 9+ immutable map
Map<String, Integer> fixed = Map.of("A", 1, "B", 2);

```

6.2 LinkedHashMap

Maintains insertion order (or access order if constructed with `accessOrder=true` — useful for LRU caches).

```

// LRU Cache using LinkedHashMap
class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;
    LRUCache(int capacity) {
        super(capacity, 0.75f, true); // accessOrder = true
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

```



```

LRUCache<Integer, String> cache = new LRUCache<>(3);
cache.put(1, "A"); cache.put(2, "B"); cache.put(3, "C");
cache.get(1); // access 1 — moves to end
cache.put(4, "D"); // evicts 2 (least recently used)
System.out.println(cache.keySet()); // [3, 1, 4]

```

6.3 TreeMap

Attribute	Detail
Backing store	Red-Black tree
Order	Natural key order or Comparator
Performance	O(log n) get/put
When to use	Sorted keys, range queries, predecessor/successor lookups

```

TreeMap<String, Integer> map = new TreeMap<>();
map.put("banana", 2);
map.put("apple", 5);
map.put("cherry", 1);
map.put("date", 3);

System.out.println(map.firstKey()); // apple
System.out.println(map.lastKey()); // date
System.out.println(map.floorKey("c")); // cherry
System.out.println(map.ceilingKey("b")); // banana
System.out.println(map.subMap("b", "d")); // {banana=2,
cherry=1}
System.out.println(map.headMap("c")); // {apple=5, banana=2}
System.out.println(map.tailMap("c")); // {cherry=1, date=3}

```

6.4 Map Comparison

Feature	HashMap	LinkedHashMap	TreeMap
Order	None	Insertion/access	Sorted
get/put	O(1) avg	O(1) avg	O(log n)
Null key	One allowed	One allowed	Not allowed
Range queries	No	No	Yes
LRU cache	No	Yes	No
Typical choice	Default	Order matters	Sorted iteration

7. Collections Utility Class

java.util.Collections provides static helper methods that operate on Collection types.

7.1 Sorting & Searching

```
List<Integer> nums = new ArrayList<>(List.of(3, 1, 4, 1, 5, 9, 2, 6));
Collections.sort(nums); // [1, 1, 2, 3, 4, 5, 6, 9]
Collections.sort(nums, Comparator.reverseOrder()); // descending
// Binary search (list must be sorted first)
Collections.sort(nums);
int idx = Collections.binarySearch(nums, 5); // index of 5
Collections.shuffle(nums); // random order
Collections.reverse(nums); // in-place reversal
Collections.rotate(nums, 2); // rotate right by 2
System.out.println(Collections.min(nums)); // minimum element
System.out.println(Collections.max(nums)); // maximum element
System.out.println(Collections.frequency(nums, 1)); // count of 1s
```

7.2 Wrappers

```
// Unmodifiable view — throws UnsupportedOperationException
on write
List<String> uList = Collections.unmodifiableList(new ArrayList<>(List.of("a", "b")));
Map<String, Integer> uMap = Collections.unmodifiableMap(new HashMap<>());
// Synchronized wrappers (coarse-grained; prefer ConcurrentHashMap for real concurrency)
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
// Singleton / empty collections (immutable)
List<String> single = Collections.singletonList("only");
Set<Integer> empty = Collections.emptySet();
// Fill and copy
```

```

List<String> filled = new ArrayList<>(Collections.nCopies(5,
"x")); // [x,x,x,x,x]

Collections.fill(filled, "z"); // replace all with z

List<String> dest = new ArrayList<>(Arrays.asList("a","b","c"));

Collections.copy(dest, List.of("X","Y","Z")); // dest must be >=
src size

```

8. Stream API

Java 8 introduced the Stream API, enabling functional-style bulk operations on collections. Streams are lazy — intermediate operations only execute when a terminal operation is invoked.

8.1 Creating Streams

```

// From collection
List<String> names = List.of("Alice", "Bob", "Carol", "Dave");
Stream<String> s1 = names.stream();

// From array
Stream<Integer> s2 = Arrays.stream(new Integer[]{1, 2, 3});

// From values
Stream<String> s3 = Stream.of("x", "y", "z");

// Infinite streams
Stream<Integer> nats = Stream.iterate(0, n -> n + 1); // 0,1,2,...
Stream<Double> randoms = Stream.generate(Math::random);

// Primitive streams (avoid boxing overhead)
IntStream ints = IntStream.range(1, 6); // 1,2,3,4,5
LongStream longs = LongStream.rangeClosed(1, 5); // 1,2,3,4,5
DoubleStream doubles = DoubleStream.of(1.1, 2.2);

```

8.2 Intermediate Operations (lazy)

```

List<String> names = List.of("Alice", "Bob", "Carol", "Anna",
"Dave");

// filter — keep matching elements
names.stream().filter(n -> n.startsWith("A")).forEach(System.out::println);

// Alice, Anna

// map — transform each element

```

```

names.stream().map(String::toLowerCase).forEach(System.out::println);
// flatMap — flatten nested streams
List<List<Integer>> nested = List.of(List.of(1,2), List.of(3,4));
nested.stream().flatMap(Collection::stream).forEach(System.out::println);
// 1234
// sorted
names.stream().sorted().forEach(System.out::println);
names.stream().sorted(Comparator.comparingInt(String::length)).forEach(System.out::println);
// distinct, limit, skip
Stream.of(1,2,2,3,3,4).distinct().forEach(System.out::print); // 1234
Stream.iterate(1, n->n+1).limit(5).forEach(System.out::print); //
12345
Stream.iterate(1, n->n+1).skip(3).limit(3).forEach(System.out::print);
// 456
// peek — for debugging (does not consume stream)
names.stream().peek(n -> System.out.println("Processing: " +
n)).count();

```

8.3 Terminal Operations

```

List<Integer> nums = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);
// count, sum, min, max, average
long count = nums.stream().filter(n -> n % 2 == 0).count(); // 5
int sum = nums.stream().mapToInt(n -> n).sum(); // 55
OptionalInt min = nums.stream().mapToInt(n -> n).min(); // 1
double avg = nums.stream().mapToInt(n -> n).average().orElse(0);
// 5.5
// reduce — fold elements into one
int product = nums.stream().reduce(1, (a, b) -> a * b); // 3628800
// collect — most versatile terminal op
List<Integer> evens = nums.stream().filter(n->n%2==0).collect(Collectors.toList());
Set<Integer> evenSet = nums.stream().filter(n->n%2==0).collect(Collectors.toSet());
String joined = List.of("a","b","c").stream().collect(Collectors.joining(",
", "[", "]"));
// [a, b, c]

```

```

// groupingBy
List<String> words = List.of("cat", "car", "bar", "bat", "bus");
Map<Character, List<String>» byFirst =
words.stream().collect(Collectors.groupingBy(w -> w.charAt(0)));
// {c=[cat, car], b=[bar, bat, bus]}
// partitioningBy (special case of grouping: true/false)
Map<Boolean, List<Integer>» parts =
nums.stream().collect(Collectors.partitioningBy(n -> n % 2 == 0));
// anyMatch, allMatch, noneMatch
boolean anyEven = nums.stream().anyMatch(n -> n % 2 == 0); //
true
boolean allPos = nums.stream().allMatch(n -> n > 0); // true
boolean noneNeg = nums.stream().noneMatch(n -> n < 0); // true
// findFirst / findAny
Optional<Integer> first = nums.stream().filter(n -> n > 5).find-
First(); // 6

```

Streams are single-use: once a terminal operation is called, the stream is exhausted. Create a new stream for each pipeline.

8.4 Parallel Streams

```

// Parallel stream — automatically splits work across ForkJoinPool
long count = IntStream.rangeClosed(1, 1_000_000)
.parallel()
.filter(n -> n % 2 == 0)
.count();
System.out.println(count); // 500000
// Convert sequential to parallel
List<Integer> big = new ArrayList<>();
// ... fill with millions of elements ...
int sum = big.parallelStream().mapToInt(Integer::intValue).sum();
// WARNING: parallel streams are only faster when:
// 1. Data is large (tens of thousands of elements minimum)
// 2. Operations are stateless and order-independent

```

// 3. Data source is splittable (ArrayList yes, LinkedList poor)

Use parallel() judiciously. For small data sets or I/O-heavy operations, parallel streams add overhead without benefit. Always benchmark.

9. Choosing the Right Collection

Need	Recommended Class	Key reason
Ordered list, fast random access	ArrayList	O(1) get by index
Frequent head/tail insert-remove	ArrayDeque	O(1) at both ends; faster than ArrayList
Queue (FIFO)	ArrayDeque / LinkedBlockingQueue	ArrayDeque is fastest non-blocking
Stack (LIFO)	ArrayDeque	Faster than java.util.Stack
Priority ordering	PriorityQueue	Min-heap; O(log n) enqueue/dequeue
Fast membership test, no order	HashSet	O(1) contains
Ordered iteration of unique elements	LinkedHashSet	Insertion order + no duplicates
Sorted unique elements / range queries	TreeSet	O(log n); natural sort
Fast key-value lookup	HashMap	O(1) average get/put
Predictable Map iteration order	LinkedHashMap	Insertion or access order
Sorted Map / range queries on keys	TreeMap	O(log n); NavigableMap methods
LRU cache	LinkedHashMap	Override removeEldestEntry
Thread-safe Map	ConcurrentHashMap	Segment locking; high concurrency
Thread-safe List (read-heavy)	CopyOnWriteArrayList	Lock-free reads
Immutable collection (Java 9+)	List.of / Map.of	Compact, null-hostile, truly immutable

10. Common Pitfalls & Best Practices

10.1 Modifying a collection while iterating

```
// BAD — throws ConcurrentModificationException
List<Integer> list = new ArrayList<>(List.of(1, 2, 3, 4, 5));
for (Integer n : list) {
    if (n % 2 == 0) list.remove(n); // modifying during enhanced for-loop
}

// GOOD — use removeIf (Java 8+)
list.removeIf(n -> n % 2 == 0);

// GOOD — use Iterator explicitly
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove(); // safe removal
}
```

10.2 equals() and hashCode() in collections

```
// BAD — custom class without equals/hashCode breaks Set/Map
class Point { int x, y; }

Set<Point> set = new HashSet<>();

set.add(new Point(1, 2));

System.out.println(set.contains(new Point(1, 2))); // FALSE —
identity, not value!

// GOOD — use record (Java 14+) or implement equals/hashCode
record Point(int x, int y) {} // equals & hashCode auto-generated

Set<Point> set2 = new HashSet<>();

set2.add(new Point(1, 2));

System.out.println(set2.contains(new Point(1, 2))); // TRUE
```

10.3 Arrays.asList() vs List.of()

```
// Arrays.asList — fixed size but MUTABLE (can set, not
add/remove)

List<String> asList = Arrays.asList("a", "b", "c");

asList.set(0, "x"); // OK

asList.add("d"); // UnsupportedOperationException

// List.of — truly IMMUTABLE (no set/add/remove, no nulls)

List<String> immutable = List.of("a", "b", "c");

immutable.set(0, "x"); // UnsupportedOperationException

// Mutable copy from immutable

List<String> mutable = new ArrayList<>(List.of("a", "b", "c"));

mutable.add("d"); // OK
```

10.4 Initial capacity

```
// BAD — default capacity 16; triggers multiple resize/copy operations

List<Integer> list = new ArrayList<>();

for (int i = 0; i < 10_000; i++) list.add(i);

// GOOD — pre-size to avoid O(n) copy operations during growth

List<Integer> list2 = new ArrayList<>(10_000);

// Same for HashMap — default load 0.75; resizes at 0.75 * capacity
```

```
Map<String, Integer> map = new HashMap<>(128); // pre-size
to avoid rehashing
```

Rule of thumb: if you know the approximate size at creation time, pass it as the initial capacity.

11. Quick-Reference: Complexity Summary

Class	Access	Search	Insert	Delete	Order
ArrayList	O(1)	O(n)	O(1)*	O(n)	Insertion
LinkedList	O(n)	O(n)	O(1)**	O(1)**	Insertion
HashSet	N/A	O(1)	O(1)	O(1)	None
LinkedHashSet	N/A	O(1)	O(1)	O(1)	Insertion
TreeSet	N/A	O(log n)	O(log n)	O(log n)	Sorted
HashMap	O(1)	O(1)	O(1)	O(1)	None
LinkedHashMap	O(1)	O(1)	O(1)	O(1)	Insertion
TreeMap	O(log n)	O(log n)	O(log n)	O(log n)	Sorted
PriorityQueue	O(n)	O(n)	O(log n)	O(log n)	Heap
ArrayDeque	O(n)	O(n)	O(1) ends	O(1) ends	Insertion

* *ArrayList insert: O(1) amortised at tail; O(n) elsewhere.* ** *LinkedList: O(1) once node is found; O(n) to find by index.*

12. Further Reading

The Java Tutorials — Collections trail: <https://docs.oracle.com/javase/tutorial/collections>

Effective Java (3rd ed.) — Joshua Bloch — Items 43-58 cover collections and streams

Java Generics and Collections — Maurice Naftalin & Philip Wadler (2006)

JEP 431: Sequenced Collections (Java 21) — <https://openjdk.org/jeps/431>